



Submitted by,

Course Title: Database Management Lab

Course Code: CSE 2302

Lab Report: 02

Student Name: Mahinur Rahman Mahin

Student ID: 242014165

Section: 01

Department: CSE

University of Liberal Arts Bangladesh (ULAB)

Submitted to,

Fatema Khan, Lecturer

Department of Computer Science & Engineering (CSE)

University of liberal arts Bangladesh

1. SQL Syntax

```
SELECT * FROM table_name;
```

Selects all records from the specified table.

2. SELECT Syntax

```
SELECT column1, column2, ...  
FROM table_name ;
```

SELECT → Specifies which data (columns) you want to retrieve. column1,column2, ...
→ The names of the columns you want to display. FROM → Specifies the table from which the data will be retrieved. table_name → The name of the table containing the data.

3. SELECT DISTINCT Statement

```
SELECT DISTINCT column1, column2, ...  
FROM table_name;
```

SELECT DISTINCT is used to retrieve only unique (distinct) values from one or more columns. It removes duplicate values from the result.

4. WHERE Clause

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

The WHERE clause is used to filter records. It retrieves only those records that satisfy a specified condition.

5. ORDER BY Keyword

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column1, column2, ... ASC|DESC;
```

The ORDER BY keyword is used to sort the result-set in ascending or descending order. The ORDER BY keyword sorts the result-set in ascending order (ASC) by default.

6. AND Operator

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 AND condition2 AND condition3 ...;
```

The WHERE clause can contain one or many AND operators. The AND operator is used to filter records based on more than one condition.

7. OR Operator

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 OR condition2 OR condition3 ...;
```

The WHERE clause can contain one or more OR operators. The OR operator is used to filter records based on more than one condition. The OR operator displays a record if any of the conditions are TRUE.

8. NOT Operator

```
SELECT column1, column2, ...  
FROM table_name  
WHERE NOT condition;
```

The NOT operator is used in the WHERE clause to return all records that DO NOT match the specified criteria. It reverses the result of a condition from true to false and vice-versa.

9. The SQL INSERT INTO Statement

i. Syntax 1

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

Specify both the column names and the values to be inserted:

ii. Syntax 2

```
INSERT INTO table_name  
VALUES (value1, value2, value3, ...);
```

If you insert values for ALL the columns of the table, you can omit the column names. However, the order of the values must be in the same order as the columns in the table:

10. NULL Values

i. IS NULL

```
SELECT Name  
FROM Student  
WHERE Phone IS NULL;
```

IS NULL is used to retrieve records where a column contains a NULL (missing) value. Retrieves students whose Phone value is NULL (empty/missing).

ii. IS NOT NULL

```
SELECT Name  
FROM Student  
WHERE Phone IS NOT NULL;
```

IS NOT NULL is used to retrieve records where a column contains a value (not NULL). Retrieves students whose Phone value is not NULL.

11. UPDATE Statement

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

The UPDATE statement is used to update or modify one or more records in a table.

12. DELETE Statement

```
DELETE FROM table_name WHERE condition;
```

The DELETE statement is used to delete one or more existing records from a table. The WHERE clause specifies which record(s) to delete.

13. Aggregate Functions

- i. COUNT() → Counts the number of rows.
- ii. SUM() → Returns the total sum of values.
- iii. AVG() → Returns the average value.
- iv. MIN() → Returns the smallest value.
- v. MAX() → Returns the largest value.

Aggregate functions are used to perform calculations on a set of values and return a single result.

14. LIKE Operator

```
SELECT column1, column2, ...  
FROM table_name  
WHERE columnN LIKE pattern;
```

The LIKE operator is used to search for values that match a specified pattern in a column.

15. Wildcards

```
SELECT column1, column2, ...  
FROM table_name  
WHERE column_name LIKE 'pattern';
```

LIKE is used with wildcards (% , _) to search for a specified pattern in a column.

16. IN Operator

```
SELECT column_name(s)  
FROM table_name  
WHERE column_name IN (value1, value2, ...);
```

The IN operator is used to select records where a column matches any value in a specified list.

17. BETWEEN Operator

```
SELECT column_name(s)  
FROM table_name  
WHERE column_name BETWEEN value1 AND value2;
```

The BETWEEN operator is used to retrieve records with values within a specified range (from value1 to value2, inclusive).

18. Aliases

i. Column Alias Syntax

```
SELECT column_name AS alias_name  
FROM table_name;
```

A column alias is used to give a temporary name to a column.

ii. Table Alias Syntax

```
SELECT column_name(s)
FROM table_name AS alias_name;
```

A table alias is used to give a temporary name to a table.

19. GROUP BY Statement

```
SELECT column1, aggregate_function(column2), column3, ...
FROM table_name
WHERE condition
GROUP BY column1, column3
ORDER BY column_name;
```

The GROUP BY clause is used to group rows that have the same values in one or more columns. It is often used with aggregate functions such as COUNT(), SUM(), AVG(), MIN(), and MAX().

20. HAVING Clause

```
SELECT column1, aggregate_function(column2), column3, ...
FROM table_name
WHERE condition
GROUP BY column1, column3
HAVING condition
ORDER BY column_name;
```

The HAVING clause is used to filter groups after the GROUP BY operation. It is mainly used with aggregate functions such as COUNT(), SUM(), AVG(), MIN(), and MAX().

21. EXISTS Operator

```
SELECT column_name(s)
FROM table_name
WHERE EXISTS (subquery);
```

The EXISTS operator is used to check whether a subquery returns any rows. If the subquery returns at least one row, EXISTS returns TRUE; otherwise, it returns FALSE.

22. ANY Operator

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ANY (subquery);
```

The ANY operator is used to compare a value with the values returned by a subquery. It returns TRUE if at least one value satisfies the specified condition.

23. ALL Operator

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ALL (subquery);
```

The ALL operator is used to compare a value with all values returned by a subquery. It returns TRUE only if every value satisfies the specified condition.